AutoVaultLK: Build and Test Instructions

How to build the project and run every test suite from a clean checkout.

1. Prerequisites

- Node.js 22 and npm
- Docker Desktop (Postgres container, needed for integration and e2e suites)
- k6 (performance tests only): https://k6.io/docs/get-started/installation/
- Copy each service's `.env.example` to `.env` (root and per service). Groq and SMTP/SES keys are optional; without them search uses the rules-based parser and emails are logged.

2. Install and build

Run in each of: `auth-user-service`, `marketplace-service`, `admin-service`, `notification-service`, `ingestion-service`, `web-frontend`:

```
npm ci
npm run build
```

3. Unit tests (no database needed)

```
# in each backend service folder
npm run test:ci

# api-gateway
cd api-gateway; npm ci; npm run test:ci; npm run test:nginx

# web-frontend (Vitest)
cd web-frontend; npm test
```

Service	Test location
auth-user-service	tests/
marketplace-service	test/unit/
admin-service	test/unit/
notification-service	test/unit/, src/**/*.spec.ts
ingestion-service	test/unit/
api-gateway	tests/*.test.js
web-frontend	src/test/

- Add `npm run test:cov` in a service to produce a coverage report (`coverage/`).
- The first run compiles TypeScript and can take a few minutes on a cold cache.

4. Integration, contract and API e2e tests (need Postgres)

```
docker compose up -d postgres
cd database; npm ci; npm run migration:run; npm run grants; npm run seed:dictionaries; npm run seed:vehicles
```

Then, in the relevant service folder:

```
npm run test:integration      # auth-user, marketplace, admin, notification, ingestion
npm run test:contract        # auth-user, admin, notification, ingestion
npm run test:e2e             # every backend service
npm run test:security        # auth-user-service only
```

5. Browser end-to-end tests (Playwright)

Start the full stack first (`docker compose up`, or the services via `scripts/start-all.ps1`), then:

```
cd e2e
npm ci
npx playwright install
npm test
npm run report        # open the HTML report
```

6. Performance tests (k6)

Start Postgres, seed the database (section 4), then build and start auth-user-service (port 3001) and marketplace-service (port 3002) with npm run start:prod. Then:

```
k6 run performance\scripts\listing-detail-baseline.js
k6 run performance\scripts\nl-search-baseline.js
k6 run performance\scripts\login-favourites-load.js   # 50 users, about 3.5 minutes
```

- Default target is `http://localhost:3002`; override with `-e BASE_URL=...`.
- Pass criteria: p95 response time under 500 ms and under 1% failed requests.
- Other scripts in `performance/scripts/` need additional services (ingestion, admin, notification) and fixtures from `performance/tools/generate-e2e-fixtures.sh`.
- The same campaigns run on GitHub via Actions: **Load Test (manual)** and the other `load-test-*.yml` workflows.

7. CI

Workflows in .github/workflows/ run each service's build and tests on push, plus Terraform validation and the production deploy.

8. Files

- Test sources: all `test/`, `tests/`, `src/test/` folders and `*.spec.ts` files, `e2e/`, `performance/`
- Runner config: each service's `package.json`, jest configs (`test/jest-e2e.json`, `test/integration/`, `test/contract/`), `e2e/playwright.config.ts`
- Build/test helpers: `Makefile`, `scripts/`
- CI: `.github/workflows/`
- Environment templates: every `.env.example`

Not included: node_modules, dist, coverage, test-results, .env.